

Preguntas del análisis

1. Justificar las jerarquías. Explique cómo organiza los elementos comunes y particulares de los vehículos y de los clientes. Indique qué clases no deberían poder instanciarse directamente y por qué.

La clase Vehiculo reúne los datos comunes: placa, marca, modelo, tarifa diaria, estado y días acumulados. Sus clases hijas, Automovil, Motocicleta, CamionetaCarga y Microbus, agregan sus características y reglas propias.

De forma similar, Cliente guarda el identificador, nombre y licencias. ClienteIndividual establece las reglas del DPI, descuento y límite de alquileres; ClienteCorporativo agrega empresa y contacto, además de sus propias condiciones.

Vehiculo y Cliente son abstractas porque representan conceptos generales. No deben instanciarse directamente: se necesita conocer el tipo concreto para determinar sus reglas.

2. Identificar los comportamientos polimórficos. Señale qué métodos declara la clase base y qué subclases los sobrescriben. Para cada uno, explique por qué la decisión corresponde a ese objeto y no a otra parte del programa.
 - calcularSubtotal(), declarado en Vehiculo, es sobrescrito por las cuatro categorías. Cada vehículo calcula su importe utilizando su tarifa y las características que determinan su recargo.
 - cumpleLicencia(), declarado en Vehiculo, es sobrescrito por las cuatro categorías porque cada una establece qué licencias acepta, considerando también si el microbús incluye piloto.
 - obtenerUmbralMantenimiento(), declarado en Vehiculo, es sobrescrito por las cuatro categorías para devolver su límite de días. Esta decisión pertenece al vehículo porque depende de su categoría.
 - obtenerCategoria(), declarado en Vehiculo, es sobrescrito por las cuatro categorías para identificar el grupo al que pertenecen en los reportes.
 - obtenerDescripcion(), implementado en Vehiculo, es sobrescrito por sus cuatro clases hijas para agregar sus características a los datos comunes.

- `calcularDescuento()`, declarado en `Cliente`, es sobrescrito por `ClienteIndividual` y `ClienteCorporativo`. El cliente determina el descuento porque este depende de sus condiciones, no del vehículo.
- `obtenerLimiteAlquileresActivos()`, declarado en `Cliente`, es sobrescrito por ambas clases hijas para indicar cuántos alquileres simultáneos permite cada tipo.
- `obtenerDescripcion()`, implementado en `Cliente`, es sobrescrito por `ClienteCorporativo` para agregar empresa y contacto. `ClienteIndividual` utiliza la descripción heredada.

3. Decidir sobre los datos repetidos. Tanto automóviles como microbuses tienen cantidad de pasajeros. Decida dónde debe ubicarse ese dato y justifique su decisión.

`cantidadPasajeros` se mantiene en `Automovil` y `Microbus`, porque en las instrucciones solo solicita registrar ese dato para esas categorías. Colocarlo en `Vehiculo` obligaría a las motocicletas y camionetas de carga a heredar información que no necesitan para este ejercicio. No se agrega otra clase intermedia únicamente para compartir ese atributo.

4. Describir un recorrido completo. Describa paso a paso cómo colaboran los objetos para confirmar un alquiler. Indique en qué momentos se ejecuta un método distinto según el tipo real del vehículo o del cliente, aunque el código que lo invoca sea el mismo.

- a. La Vista solicita la placa, el identificador del cliente y los días. El Controlador envía esos datos a `RentaMovil`.
- b. `RentaMovil` localiza al cliente y al vehículo, y valida la duración.
- c. Comprueba disponibilidad, licencias y límite de alquileres activos. Las llamadas a `cumpleLicencia()` y `obtenerLimiteAlquileresActivos()` ejecutan la versión correspondiente al tipo real de cada objeto.
- d. El vehículo calcula el subtotal mediante `calcularSubtotal()`. Luego, calcula el descuento mediante `calcularDescuento()`.
- e. Se crea una `Cotizacion` y la Vista muestra los montos y todos los impedimentos encontrados (si es que hay).
- f. Si las condiciones se cumplen, se solicita la aceptación. Si el cliente cancela, no se modifica ningún registro.
- g. Si acepta, `RentaMovil` vuelve a validar las condiciones, crea el `Alquiler`, cambia el vehículo al estado `ALQUILADO` y guarda el registro.

- h. El total del alquiler registrado se incluye en los reportes de ingresos. La llamada a los métodos es la misma, aunque su implementación cambie según el vehículo o cliente.
- 5. Evaluar la extensión. Explique qué clases tendrían que crearse o modificarse para incorporar vehículos eléctricos y clientes gubernamentales. Indique qué partes del sistema no deberían cambiar.

Para incorporar vehículos eléctricos como nueva categoría, se crearía `VehiculoElectrico`, heredando de `Vehiculo`. Definiría sus atributos y sobrescribiría los métodos de cobro, licencia, mantenimiento, categoría y descripción.

Para clientes gubernamentales, se crearía `ClienteGubernamental`, heredando de `Cliente`, con sus reglas de descuento y límite de alquileres. También podría ampliar la descripción si necesita datos adicionales.

Se modificarían las opciones de registro del `Controlador` para solicitar los nuevos datos y construir esos objetos. Si se desean ejemplos iniciales, también se actualizaría `DatosIniciales`. Las colecciones, las operaciones generales de `RentaMovil`, los reportes y los métodos de la `Vista` que muestran las listas no deberían cambiar, porque trabajan con los tipos comunes `Vehiculo` y `Cliente`.